

Document Control — Target System Architecture

Status: TARGET / FUTURE-STATE DESIGN — NOT YET IMPLEMENTED

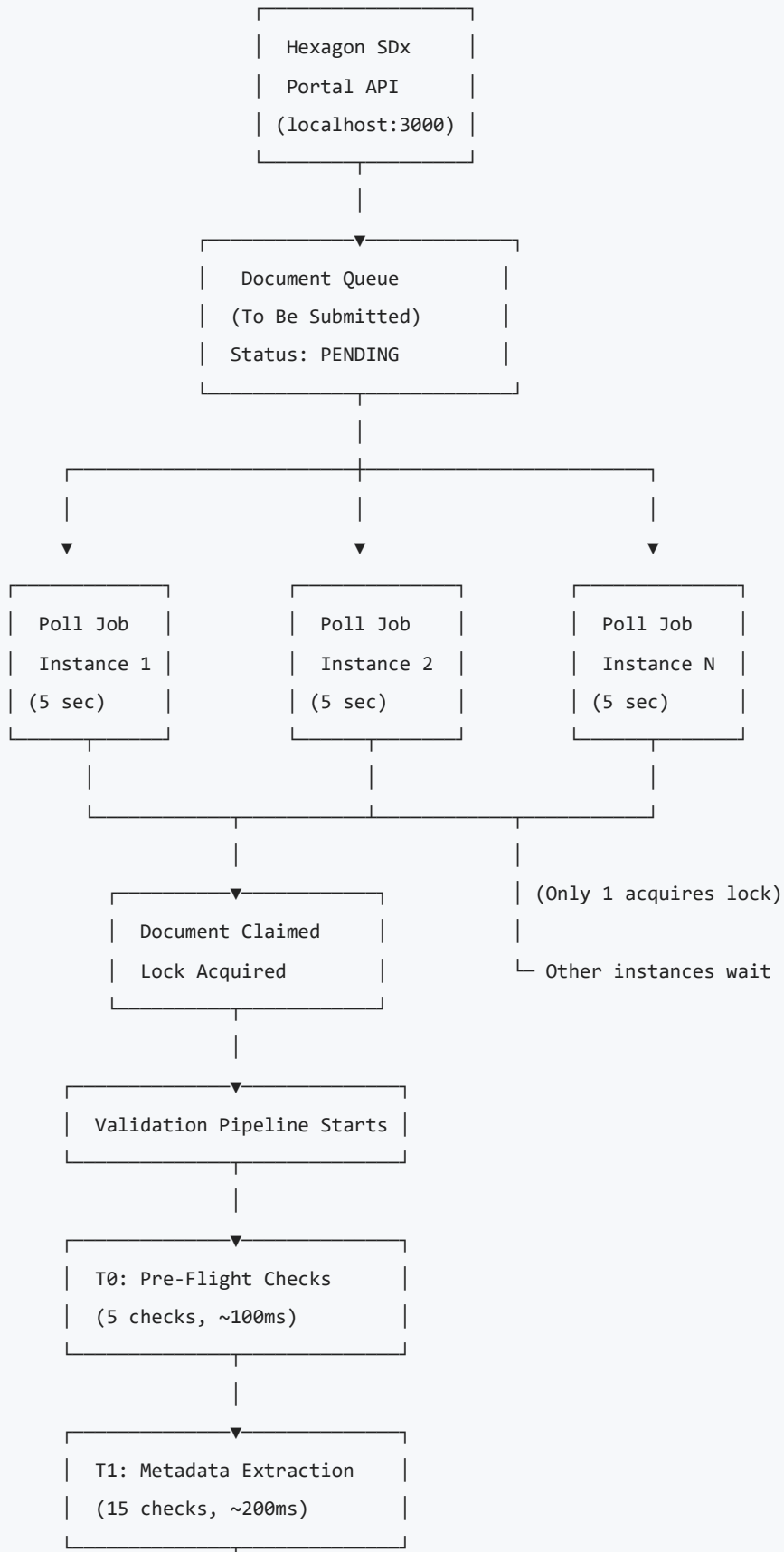
This document describes the production-grade architecture the bot is intended to evolve into. It is **not** a description of the current POC. For the current as-built system, see [SYSTEM_ARCHITECTURE.md](#). Items below such as the polling daemon, distributed locking, database schema, multi-instance coordination, checkpoint/resume, and `/api/health` endpoint are **planned work**, not running code.

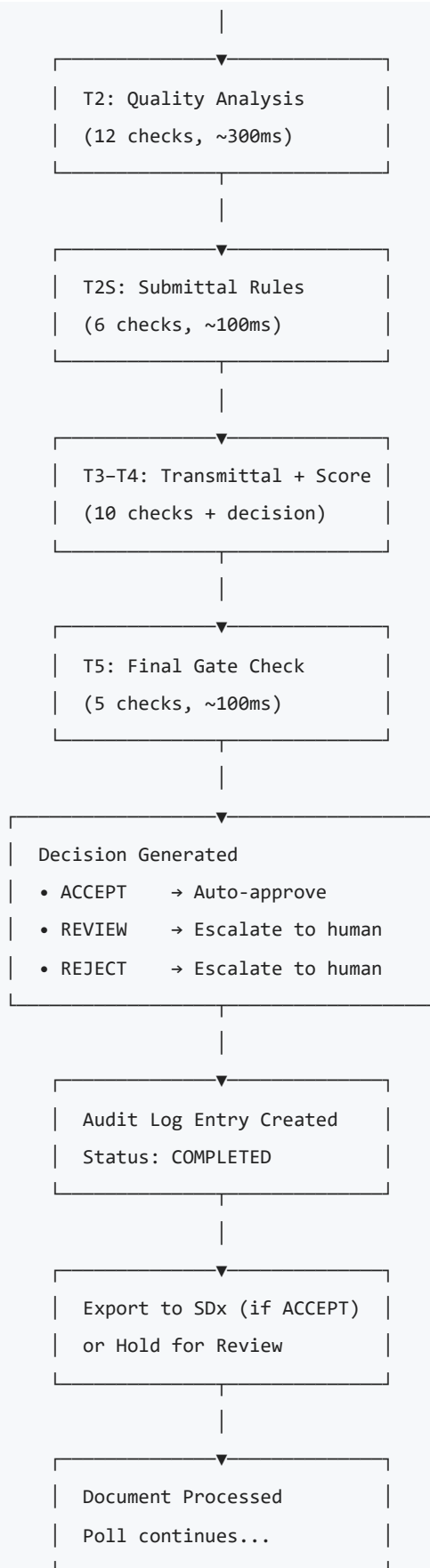
Overview

The target system uses a **polling-based job queue** architecture with resilience mechanisms for network failures, UI downtime, and processing recovery. Documents flow through multiple validation tiers with persistent state tracking.

Core Architecture Diagram

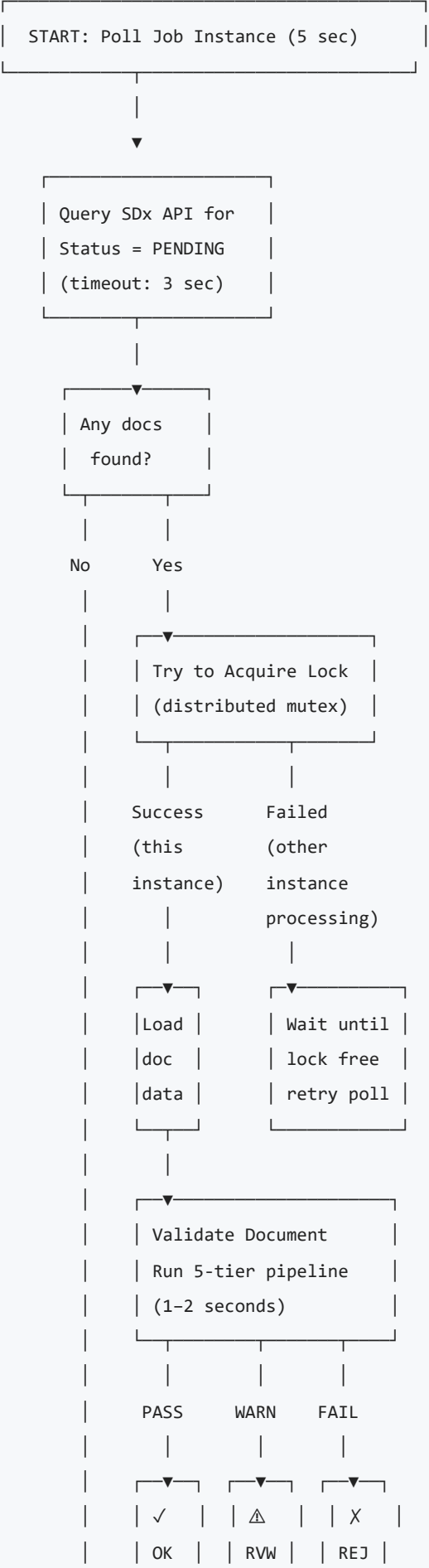
DOCUMENT LIFECYCLE FLOW

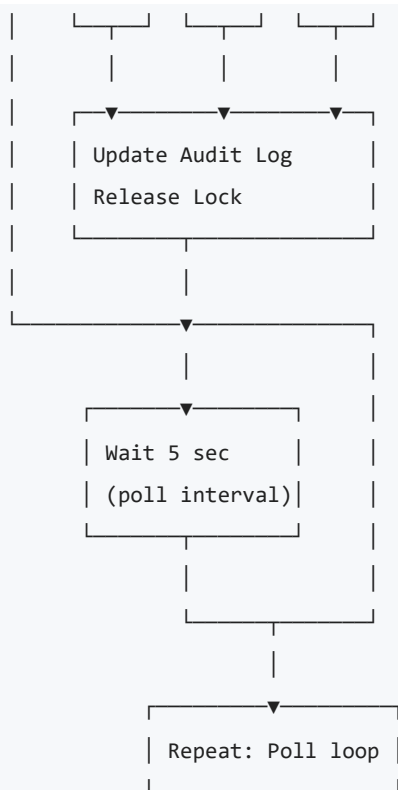




Job Polling Mechanism

Polling Flow Diagram



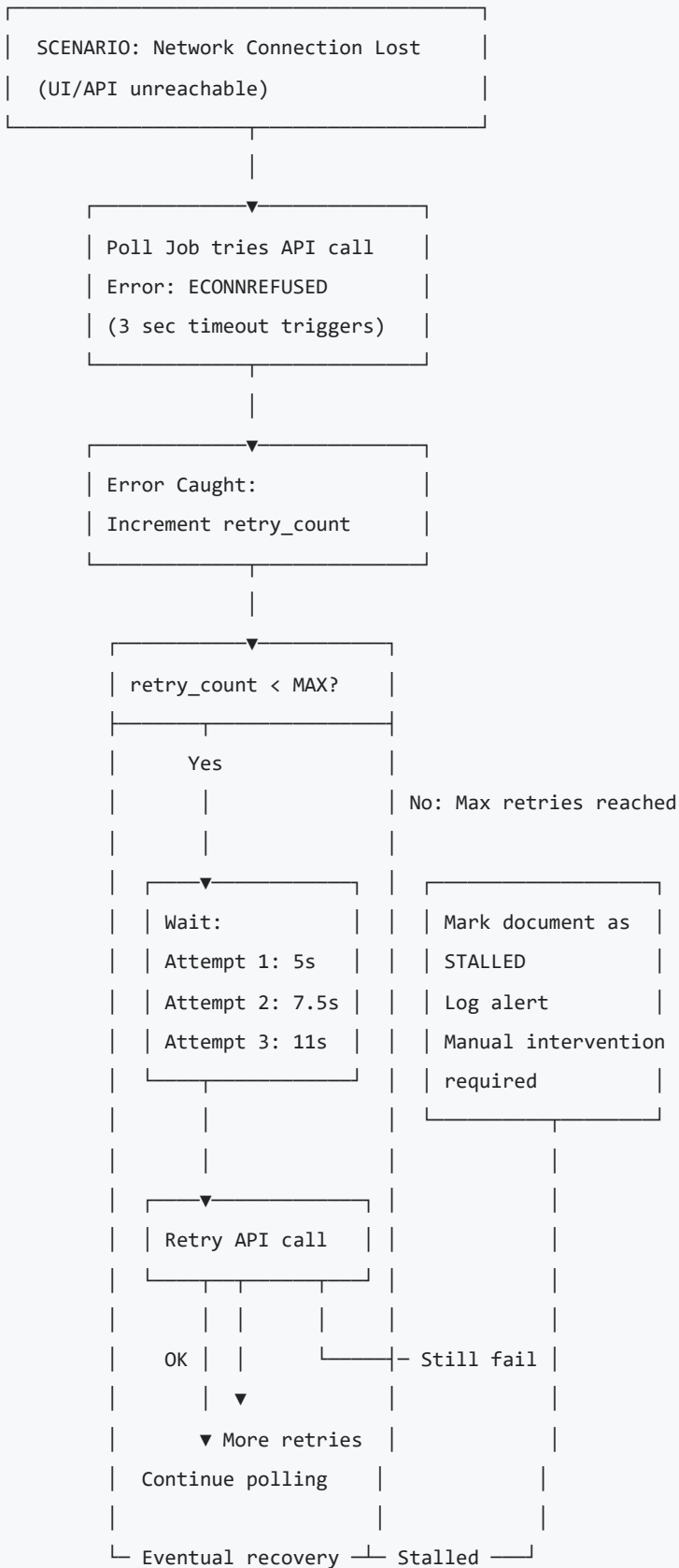


Polling Configuration

```
// Polling parameters
const POLL_INTERVAL = 5000;           // 5 seconds
const LOCK_TIMEOUT = 30000;           // 30 sec lock expiry (self-healing)
const API_TIMEOUT = 3000;             // 3 sec API call timeout
const MAX_RETRIES = 3;                // Retry on transient failure
const BACKOFF_MULTIPLIER = 1.5;      // Exponential backoff (1.5x)
```

Network Failure Handling

Network Down Scenario



State during network outage:

- └ Documents in "PENDING" remain queued
- └ Polling job keeps retrying (exponential backoff)
- └ No documents are lost

- └ Lock auto-expires after 30 seconds (other instances can retry)
- └ Dashboard can show "Network Status: OFFLINE"
- └ Recovery is automatic when network restored

Transient Error Recovery

└ Transient Errors (Retry)
• API timeout (> 3 sec)
• Connection refused (ECONNREFUSED)
• Service unavailable (HTTP 503)
• Temporary server error (HTTP 500)
• Network unreachable (ENETUNREACH)
Action: Retry with exponential backoff
Max attempts: 3
Backoff: 5s → 7.5s → 11.25s

└ Permanent Errors (Escalate)
• Unauthorized (HTTP 401)
• Forbidden (HTTP 403)
• Not found (HTTP 404)
• Validation error (app-level)
Action: Log error, mark doc FAILED
Audit trail: Record error reason
Escalate: Notify admin/user

UI Downtime Handling

UI Goes Down (API Still Up)

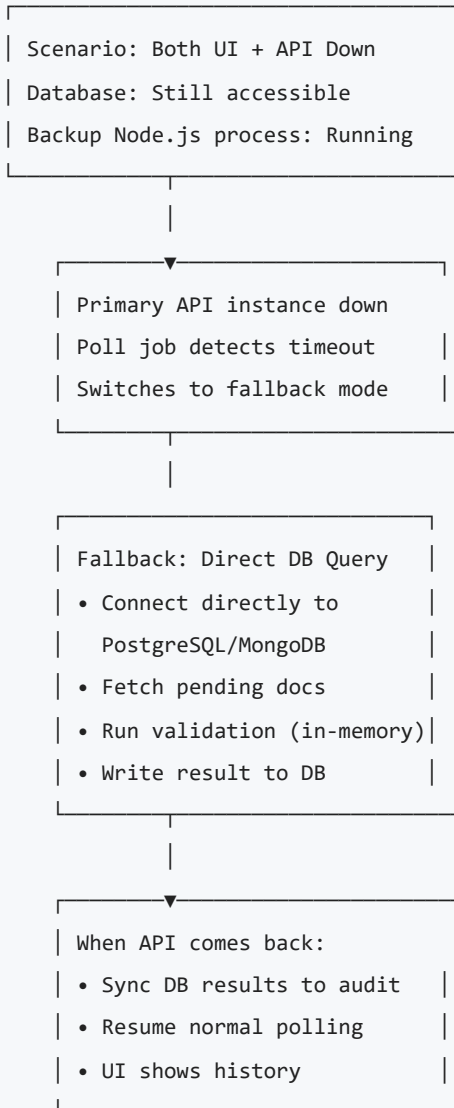
| Scenario: Dashboard UI Down |
| API Server: Still responding |
| Database: Still available |

| User opens |
| localhost:4200 |
| → Connection |
| timeout |

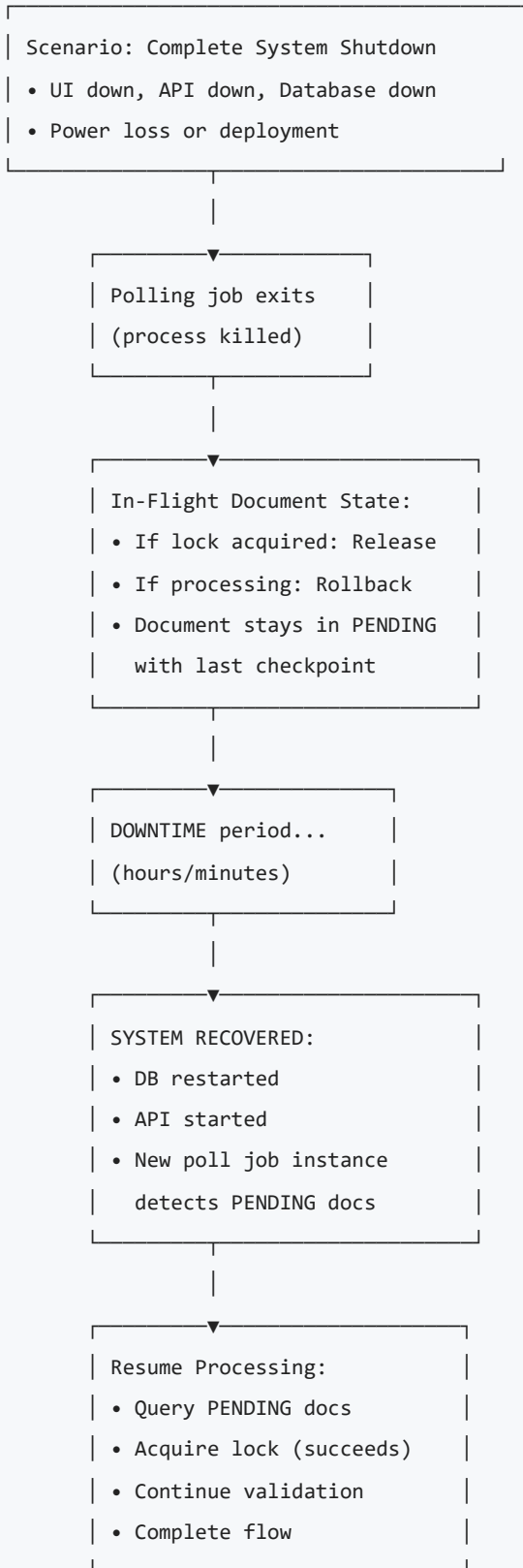
| Backend polling job still |
| running (Node.js process) |
| • Queries API |
| • Gets documents |
| • Runs validation pipeline |
| • Updates audit log |
| {...} Continues normally |

| When UI comes back online: |
| • Reconnects to API |
| • Fetches audit log history |
| • Shows all processed docs |
| • No data lost |

UI + API Down (Database Available)

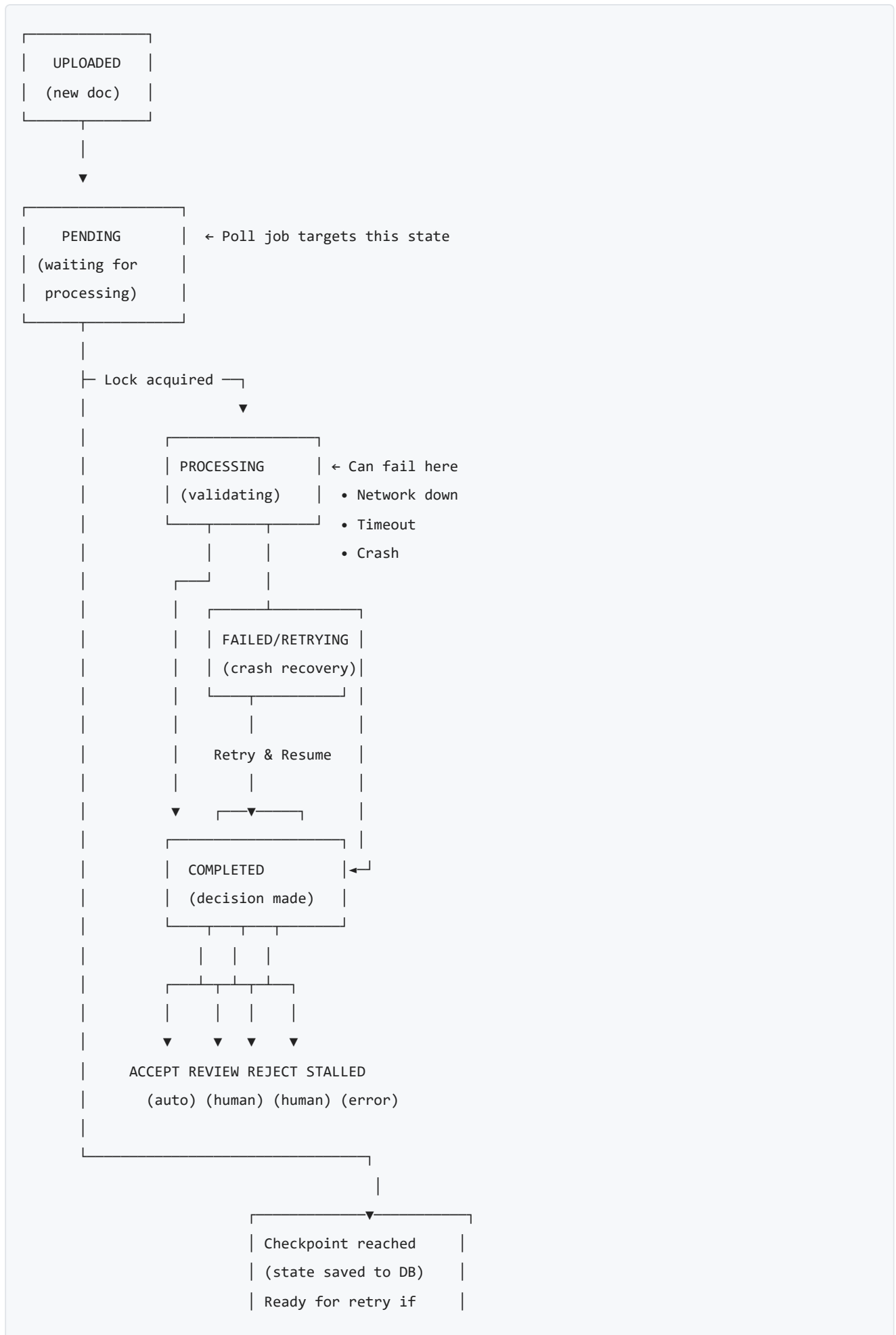


Complete System Down + Recovery



State Management & Persistence

Document State Machine



Lock Management (Distributed Mutex)

Lock Table in Database:		
document_id	lock_holder	expires_at
DOC-001	instance-A	12:03:45
DOC-002	instance-B	12:03:47
DOC-003	instance-C	12:03:42
	(auto-released)	
DOC-004	(free)	NULL

← EXPIRED

Behavior:

1. Instance tries to acquire lock:

```
UPDATE locks SET lock_holder = 'instance-X' WHERE document_id = 'DOC-001' AND (lock_holder IS NULL OR
```

2. If successful (1 row updated):

- Instance proceeds with validation
- Updates lock expires_at = NOW() + 30 seconds

3. If lock held by another instance:

- Instance waits
- Retries every 2 seconds

4. If lock expired:

- Auto-release (cascading recovery)
- Another instance acquires lock
- Processing resumes

Failure Scenario: Processing in Progress + Crash

Timeline: Document Processing Crash

```
12:03:00 | Instance-A acquires lock for DOC-001
         | (Lock expires at 12:03:30)
         |
12:03:05 | Running T0-T2 checks...
         |
12:03:15 | [CRASH] Instance-A process dies
         | (Network interrupted / Kill signal)
         |
         |─ Lock HELD by dead instance
         |─ Document state: PROCESSING
         |─ No checkpoint written
         |
12:03:20 | Instance-B queries for PENDING docs
         | Finds DOC-001 in PROCESSING
         | Tries to acquire lock
         | [WAIT] Lock still held (expires in 10 sec)
         |
12:03:30 | Lock EXPIRED (30 sec timeout elapsed)
         | Auto-released by database
         |
12:03:31 | Instance-B acquires lock successfully
         | Reads checkpoint (T0-T2 completed)
         | Resumes from T2S (where Instance-A crashed)
         | Completes validation (T3-T5)
         | Writes decision to audit log
         | Releases lock
         |
12:03:35 | Document COMPLETED
         | DOC-001 moved from PROCESSING → COMPLETED
         |
12:03:36 | Poll loop continues...
```

Result:

- ✓ No document lost
- ✓ Checkpoint saved (at least pre-flight status known)
- ✓ Recovery automatic (no manual intervention)
- ✓ Audit trail shows: instance-A started, instance-B finished

Database Schema (Locking + Checkpoint)

```

-- Main audit log
CREATE TABLE audit_log (
  id UUID PRIMARY KEY,
  document_id VARCHAR(50),
  status VARCHAR(20), -- PENDING, PROCESSING, COMPLETED
  decision VARCHAR(20), -- ACCEPT, REVIEW, REJECT
  score NUMERIC(5,2),
  pass_count INT,
  fail_count INT,
  warn_count INT,
  checkpoints JSONB, -- {T0: pass, T1: pass, T2: warn, T2S: pass, ...}
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);

-- Distributed lock
CREATE TABLE document_locks (
  document_id VARCHAR(50) PRIMARY KEY,
  lock_holder VARCHAR(100), -- instance ID
  acquired_at TIMESTAMP,
  expires_at TIMESTAMP
);

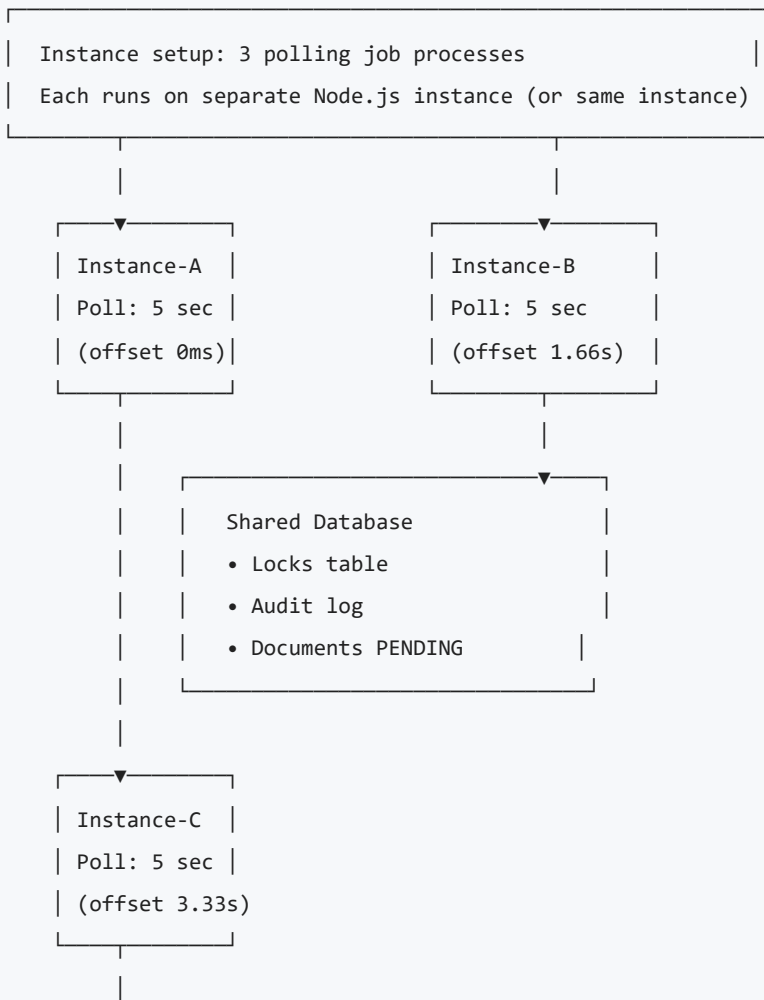
-- Checkpoint for crash recovery
CREATE TABLE processing_checkpoint (
  document_id VARCHAR(50) PRIMARY KEY,
  checkpoint_tier VARCHAR(20), -- T0, T1, T2, T2S, T3, T4, T5
  checkpoint_data JSONB, -- {metadata: {...}, quality_results: {...}}
  updated_at TIMESTAMP
);

-- Retry history
CREATE TABLE retry_history (
  id SERIAL PRIMARY KEY,
  document_id VARCHAR(50),
  attempt INT,
  status VARCHAR(20), -- success, failure, timeout
  error_message TEXT,
  attempted_at TIMESTAMP
);

```

Multi-Instance Coordination

Three Polling Instances (Load Balanced)



Staggered polling prevents thundering herd:

- Instance-A queries at: 0ms, 5000ms, 10000ms, ...
- Instance-B queries at: 1666ms, 6666ms, 11666ms, ...
- Instance-C queries at: 3333ms, 8333ms, 13333ms, ...

Result:

- ✓ Continuous coverage (always one instance polling)
- ✓ Only ONE acquires lock per document
- ✓ Others wait/retry
- ✓ Load spread across time
- ✓ If one crashes, others cover

Monitoring & Alerting

Health Check Metrics

Real-time Monitoring Dashboard	
Metric	Threshold
Polling latency	< 2s
Poll success rate	> 95%
Documents in queue	any
Processing time (avg)	< 5s
Network error rate	< 1%
Lock contention	< 10%
Database connection	healthy
Memory usage (poll job)	< 200MB

Alerts:

- **CRITICAL:**
 - Poll job process died
 - Database unreachable
 - Network down > 5 min

- **WARNING:**
 - Polling latency > 5s
 - Stalled documents > 10
 - Error rate > 5%
 - Lock stuck > 60s

- **INFO:**
 - Document processed
 - New document queued
 - Instance joined pool

Recovery Procedures

Manual Recovery (if needed)

```

# 1. Check polling status
curl http://localhost:3000/api/health
# Returns: { status: "ok", poll_active: true, documents_pending: 15 }

# 2. Check for stalled documents (locked > 5 min)
SELECT * FROM audit_log
WHERE status = 'PROCESSING'
AND updated_at < NOW() - INTERVAL '5 minutes';

# 3. Force unlock a document (CAREFUL!)
UPDATE document_locks
SET lock_holder = NULL, expires_at = NULL
WHERE document_id = 'DOC-001';

# 4. Inspect checkpoint (resume point)
SELECT * FROM processing_checkpoint
WHERE document_id = 'DOC-001';

# 5. Restart polling job
kill $(pgrep -f "polling-job.js")
npm start # Restarts with fresh lock acquisition

# 6. Monitor recovery
tail -f logs/polling-job.log

```

Summary: System Resilience

Scenario	Detection	Recovery	Time-to-Recover
Network timeout	Poll retry logic	Exponential backoff	< 30s
API service down	Connection error	Auto-fallback to DB	< 10s
UI down	N/A (backend continues)	Auto-resume on UI restart	Instant
Instance crash	Lock timeout	Auto-release (30s) + peer pickup	< 35s
Database down	Connection failure	N/A (system halts gracefully)	Manual restart needed
Stalled lock	Age > 30s	Auto-expire + reassign	~30s
Document corruption	Validation failure	Log error + manual review	Manual

